

Supplementary Material: Scalable Algorithm for Dynamic Quasi-clique Detection

I. OMITTED PROOFS

A. Proof of the Deletion Lemma

Without loss of symmetry, we assume $d_S(u) \leq d_S(v)$.

For (1), let $S' = S \setminus \{u\}$, then

$$\begin{aligned} E(S') &\geq \frac{1}{2} \left(\left[\sum_{i=1}^s d_S(a_i) \right] - 2d_S(u) \right) \\ &\geq \frac{1}{2} \left(\left[\sum_{i=1}^s d_S(a_i) \right] - d_S(u) - d_S(v) \right) \\ &\geq \frac{s-2}{2} d_S(a_1) \\ &\geq \alpha \binom{s-1}{2}. \end{aligned}$$

Therefore, we have $\delta(S') \geq \alpha$.

For (2), let $S' = S \setminus \{a_1\}$, then

$$\begin{aligned} E(S') &\geq \frac{1}{2} \left(\left[\sum_{i=1}^s d_S(a_i) \right] - 2d_S(a_1) - 2 \right) \\ &\geq \frac{1}{2} \cdot \frac{s-2}{s} \left(\sum_{i=1}^s d_S(a_i) \right) \\ &\geq \alpha \binom{s-1}{2}. \end{aligned}$$

Therefore, we have $\delta(S') \geq \alpha$.

For (3), we only need to prove the case $d_S(u) = d_S(a_1)$.

Let $S' = S \setminus \{u\}$, then

$$\begin{aligned} E(S') &\geq \frac{1}{2} \left(\left[\sum_{i=1}^s d_S(a_i) \right] - 2d_S(u) \right) \\ &\geq \frac{1}{2} \left(\sum_{i=3}^s d_S(a_i) \right) \\ &\geq \frac{1}{2} \cdot \frac{s-2}{s} \left(\sum_{i=1}^s d_S(a_i) \right) \\ &\geq \alpha \binom{s-1}{2}. \end{aligned}$$

Therefore, we have $\delta(S') \geq \alpha$.

II. IMPLEMENTATION DETAILS OF THE NEIGHBOR-SEARCH-BASED ALGORITHM

In this section, we discuss the implementation details of the three routines `Build`, `AddEdge` and `DeleteEdge` for the NSF framework.

Algorithm 1: NSF: `Build`

Input: G : A graph; α : least required density of quasi-cliques.
Output: C : A mapping from vertex u to maximal quasi-clique; S : A set of pairs.

```

1 for  $u \in V(G)$  do
2    $C(u) \leftarrow \text{Detect}(u)$ ;
3   if  $\delta(C(u)) \geq \alpha$  then  $S \leftarrow S \cup \{(|C(u)|, u)\}$  ;
4 return  $C, S$ ;
```

Algorithm 2: NSF: `AddEdge`

Input: G : A graph; α : least required density of quasi-cliques; (u, v) : the edge to add; C : A mapping from vertex u to maximal quasi-clique; S : A set of pairs; $Batch$: the number of operations to trigger rebuild. R : maximum number to re-detect.

Output: Q : A near-maximum quasi-clique.

```

1  $E(G) \leftarrow E(G) \cup \{(u, v)\}$ ;
2 if # of operations =  $Batch$  then
3    $C, S \leftarrow \text{Build}(G)$ ;
4    $(Size, w) \leftarrow \max_{x \in S} (|C(x)|, x)$ ;
5   return  $C(w)$ ;
6 for  $w \in N(u) \cup N(v)$  do
7    $\_ \leftarrow \text{Update variables that used in Detect process}$ ;
8    $S \leftarrow S \setminus \{(|C(u)|, u), (|C(v)|, v)\}$ ;
9    $C(u) \leftarrow \text{Detect}(u)$ ;
10   $C(v) \leftarrow \text{Detect}(v)$ ;
11 if  $\delta(C(u)) \geq \alpha$  then  $S \leftarrow S \cup \{(|C(u)|, u)\}$  ;
12 if  $\delta(C(v)) \geq \alpha$  then  $S \leftarrow S \cup \{(|C(v)|, v)\}$  ;
13 return  $\text{Extract}(G, \alpha, C, S, R)$ ;
```

a) *Initialization:* As shown in Algorithm 1, the initialization of NSF is as follows: We call the neighbor search routine `Detect` for each vertex u in $V(G)$, storing the result in $C(u)$ (Algorithm 1, 2). If the found quasi-clique $C(u)$'s density is at least α (Algorithm 1), we store a tuple $(|C(u)|, u)$ in a ordered set S (Algorithm 1).

b) *Insertion:* The implementation details are shown in Algorithm 2. For an insertion of edge (u, v) , we first insert (u, v) into the graph (Algorithm 2). Same as DMI, we do a batch rebuild to accommodate the changes of a large number of operations (Algorithm 2 to 5). For all neighbors of u and v , we first update information that used in `Detect` (Algorithm 2). Then, we remove the original quasi-clique information of $C(u)$ and $C(v)$ from S (Algorithm 2). Setting u and v as the center, we call `Detect` to extract a quasi-clique from their neighborhood, updating $C(u)$ and $C(v)$ (Algorithm 2, 10). If the found quasi-clique's density is at least α , we insert a

Algorithm 3: NSF : DeleteEdge

Input: G : A graph; α : least required density of quasi-cliques; (u, v) : the edge to delete; C : A mapping from vertex u to maximal quasi-clique; S : A set of pairs; $Batch$: the number of operations to trigger rebuild; R : maximum number to re-detect.

Output: Q : A near-maximum quasi-clique.

```
1  $E(G) \leftarrow E(G) \setminus \{(u, v)\}$ ;
2 if # of operations =  $Batch$  then
3    $C, S \leftarrow Build(G)$ ;
4    $(Size, w) \leftarrow \max_{x \in S} (|C(x)|, x)$ ;
5   return  $C(w)$ ;
6 for  $w \in N(u) \cup N(v)$  do
7    $\_$  Update variables that used in Detect process;
8  $S \leftarrow S \setminus \{(|C(u)|, u), (|C(v)|, v)\}$ ;
9  $C(u) \leftarrow Detect(u)$ ;
10  $C(v) \leftarrow Detect(v)$ ;
11 if  $\delta(C(u)) \geq \alpha$  then  $S \leftarrow S \cup \{(|C(u)|, u)\}$  ;
12 if  $\delta(C(v)) \geq \alpha$  then  $S \leftarrow S \cup \{(|C(v)|, v)\}$  ;
13 return Extract( $G, \alpha, C, S, R$ );
```

Algorithm 4: NSF : Extract

Input: G : A graph; α : least required density of quasi-cliques; C : A mapping from vertex u to maximal quasi-clique; S : A set of pairs; R : maximum number to re-detect.

Output: Q : A near-maximum quasi-clique.

```
1  $Q \leftarrow \emptyset$ ;
2 for  $i \leftarrow 1$  to  $R$  do
3    $(Size, w) \leftarrow \max_{x \in S} (|C(x)|, x)$ ;
4    $S \leftarrow S \setminus \{(Size, w)\}$ ;
5    $C(w) \leftarrow Detect(w)$ ;
6   if  $\delta(C(w)) < \alpha$  then continue ;
7    $S \leftarrow S \cup \{(|C(w)|, w)\}$ ;
8   if  $|C(w)| > |Q|$  then  $Q \leftarrow C(w)$  ;
9   if  $|C(w)| \geq Size$  then break ;
10 return  $Q$ ;
```

density requirement. For efficiency, we do not validate all of them during the deletion. Instead, when trying to compute a result from S , we repeatedly extract the largest quasi-clique $C(w)$ in S (Algorithm 4) and recompute the quasi-clique in the corresponding neighborhood (Algorithm 4). We take this chance to update S accordingly: We delete the original information from S (Algorithm 4). If the density is less than δ , we directly discard it (Algorithm 4). Otherwise, we insert it back to S (Algorithm 4) and try updating the current found best answer (Algorithm 4). Specifically, if we find that the newly found quasi-clique is even larger than the original stored one, we exit immediately (Algorithm 4). We repeat this process for a fixed time R and then return the current found best quasi-clique (Algorithm 4).

tuple $(|C(u)|, u)$ $(|C(v)|, v)$ into S (Algorithm 2 to 12). In the end, we call the Extract routine to return the largest one in S as the result (Algorithm 2).

c) *Deletion*: Algorithm 3 gives the implementation details of the DeleteEdge. It is almost identical to AddEdge: we first delete (u, v) from the graph (Algorithm 3). A batch rebuild is used to accommodate the changes of a large number of operations (Algorithm 3 to 5). We update all neighbors' information of u and v that is used in Detect (Algorithm 3), as well as removing the original quasi-clique information of $C(u)$ and $C(v)$ from S (Algorithm 3). We call Detect(u) and Detect(v) to extract a quasi-clique from their neighborhood, updating $C(u)$ and $C(v)$ (Algorithm 3, 10). If the found quasi-clique's density is at least α , we insert a tuple $(|C(u)|, u)$ $(|C(v)|, v)$ into S (Algorithm 3 to 12). In the end, we call Extract to compute a quasi-clique to return (Algorithm 3).

d) *Solution Computation*: The necessity of this subroutine (Algorithm 4) Extract is that edge deletion could cause some of the quasi-cliques stored in S to fail to meet the